Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps

EXPERIMENT:-5

Aim: To automate application deployment using Github/Jenkins.

LO mapped: **LO3**

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment.

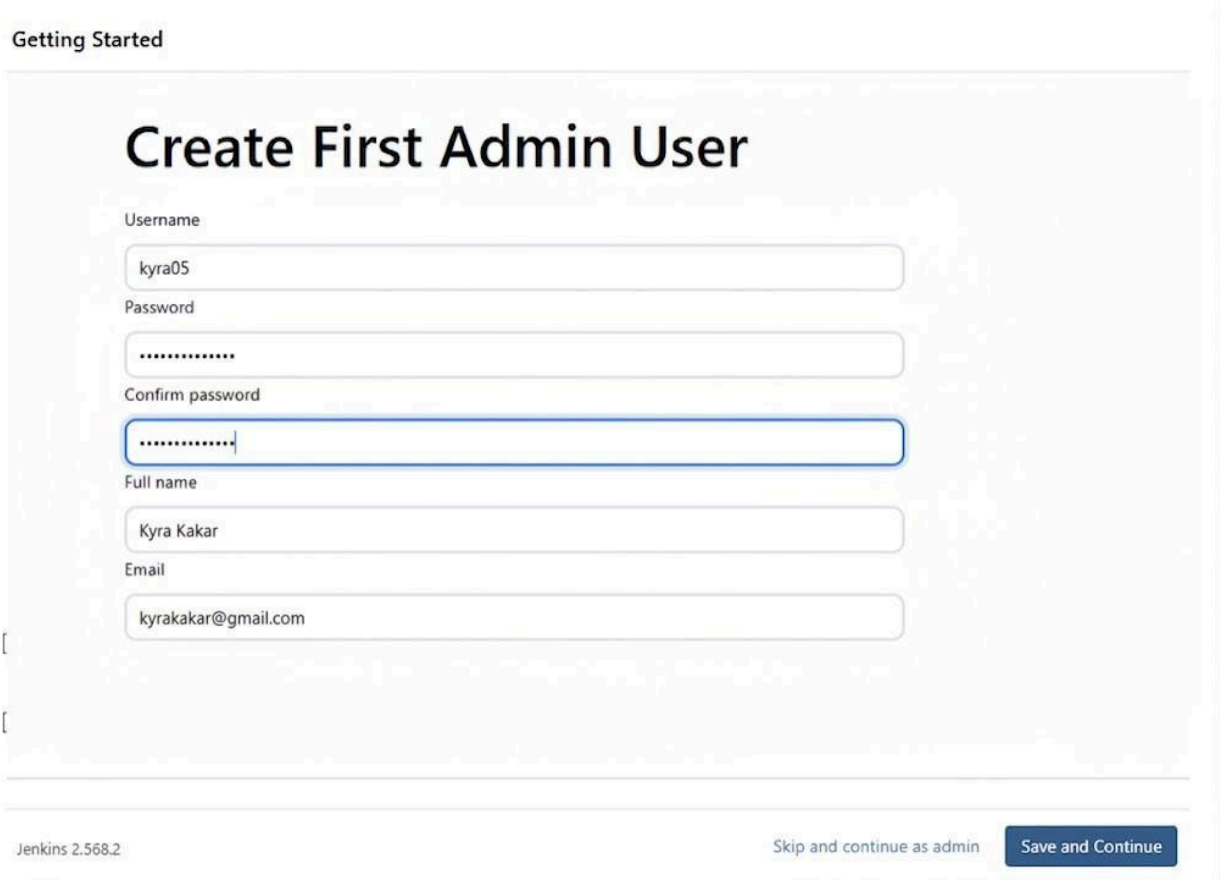
Theory:

Continuous Integration (CI) and Continuous Deployment (CD) are important DevOps practices that connect software development with deployment and operations. CI involves frequently merging code changes into a shared repository, followed by automated building and testing to identify errors early and improve software quality. CD automates the process of packaging, configuring, and deploying verified changes to staging or production environments with minimal manual intervention.

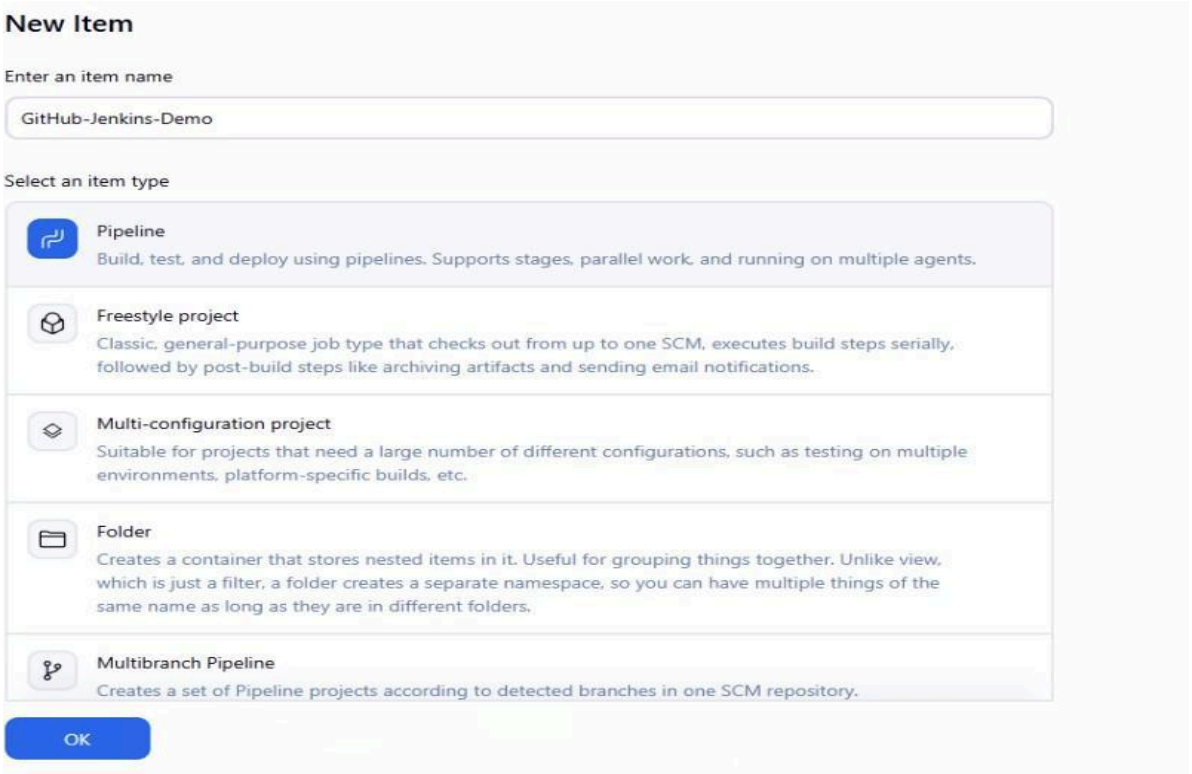
Jenkins is an open-source automation server used to automate software building, testing, and deployment. Its Declarative Pipeline allows developers to define different stages and steps using a Jenkinsfile written in a structured Groovy-based syntax. Since the Jenkinsfile is stored with the source code, the deployment process becomes version-controlled, consistent, and reproducible.

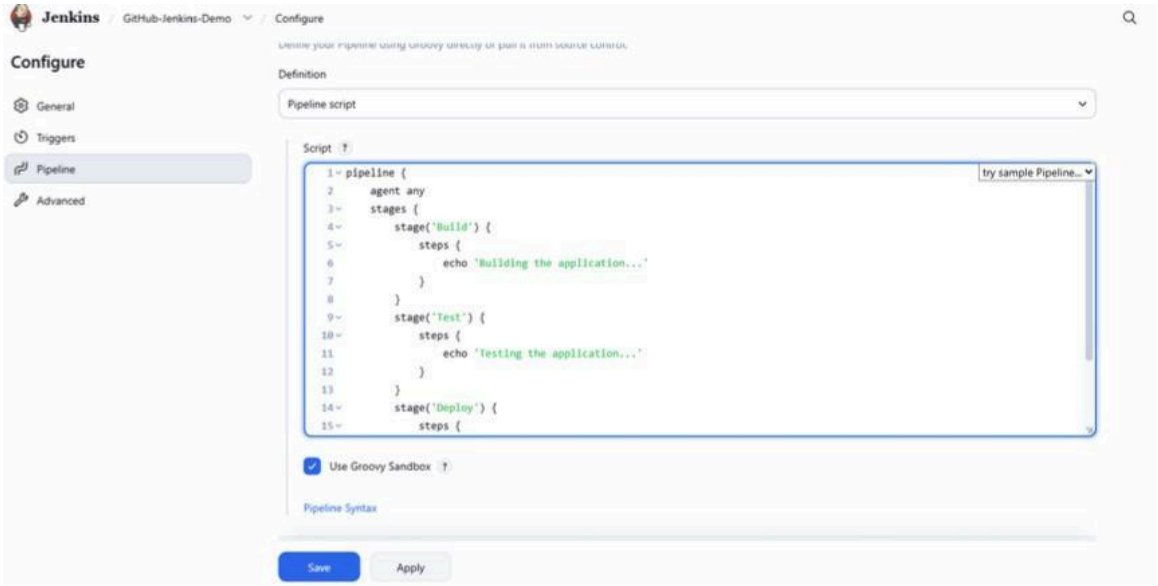
Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps

Output:

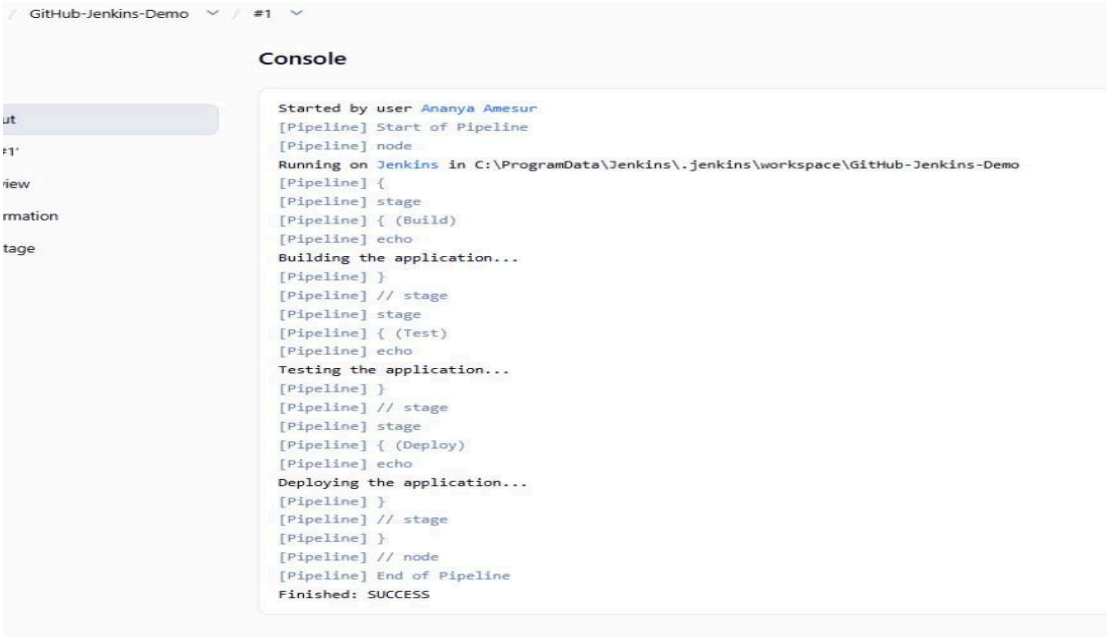


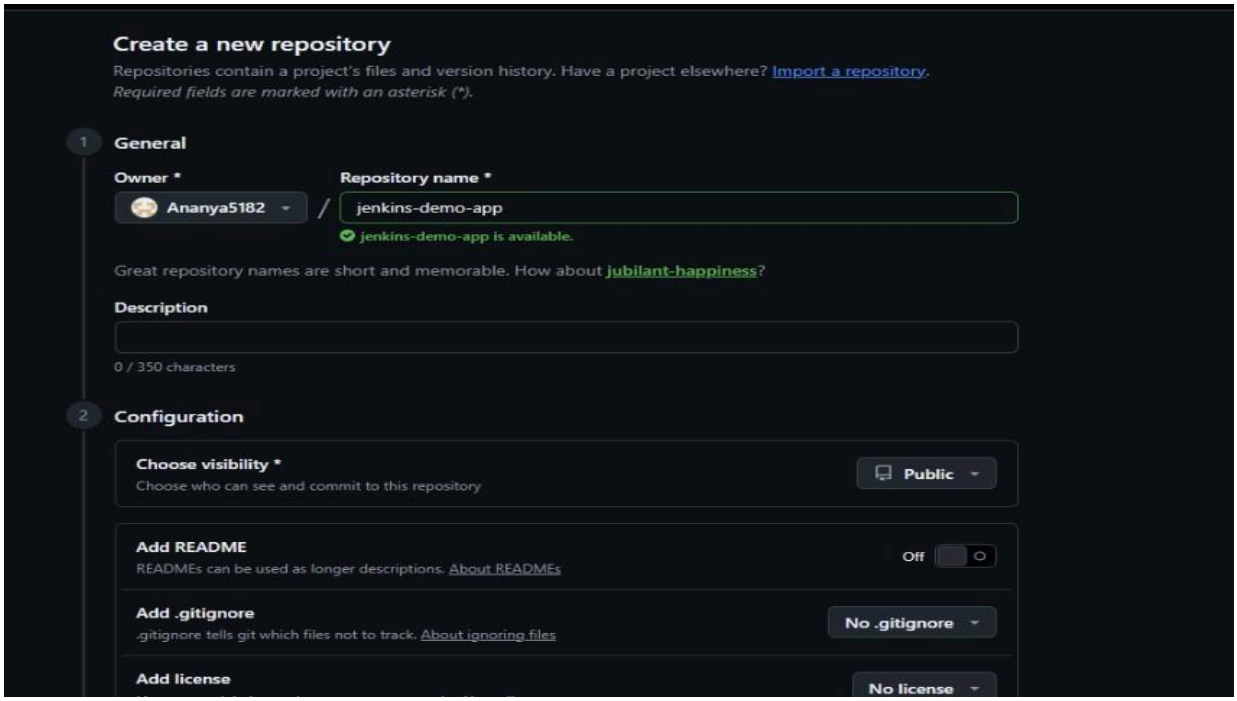
Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps





Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps





Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps

Code Blame 22 lines (22 loc) · 578 Bytes Raw

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
                bat 'if not exist "C:\\Deployments\\DemoApp" mkdir "C:\\Deployments\\DemoApp"'
                bat 'copy index.html "C:\\Deployments\\DemoApp\\index.html"'
            }
        }
    }
}
```

jenkins-demo-app / index.html in main

Cancel changes | Commit changes...

Edit | Preview

Spaces 4 No wrap

```
<!DOCTYPE html>
<html>
<head>
    <title>CI/CD Demo Application</title>
</head>
<body>
    <h1>Application Deployed Successfully via Jenkins!</h1>
</body>
</html>
```

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Ananya5182/jenkins-demo-app.git

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto)

Additional Behaviours

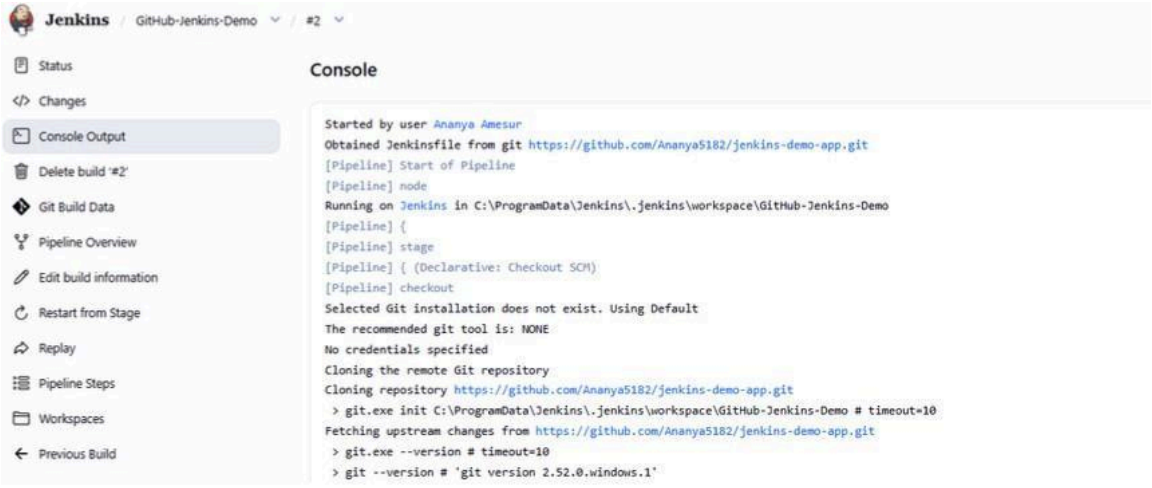
+ Add

Script Path ?

Jenkinsfile

Lightweight checkout ?

Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps




```
 > git.exe --version # timeout=10
 > git --version # 'git version 2.52.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/Ananya5182/jenkins-demo-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/Ananya5182/jenkins-demo-app.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 0c8d21cde227e66276fd9b2df19fc1815a852684 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 0c8d21cde227e66276fd9b2df19fc1815a852684 # timeout=10
Commit message: "Rename Jenkinsfile.txt to Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Building the application...
```

Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps

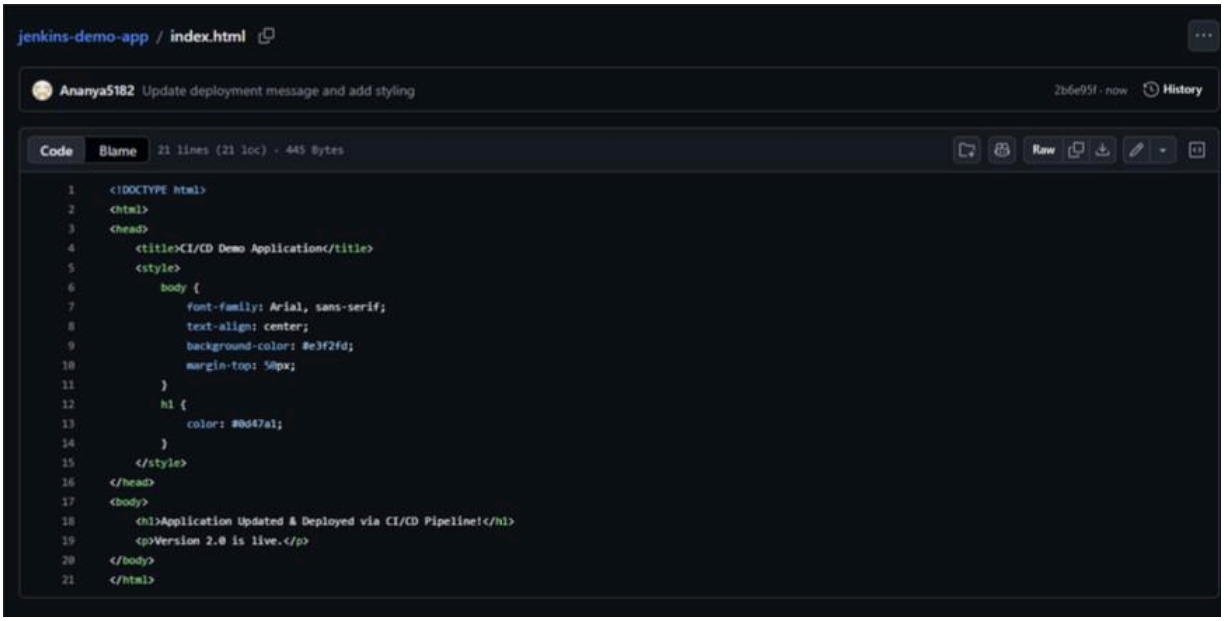
```
Building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying the application...
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>if not exist "C:\Deployments\DemoApp" mkdir "C:\Deployments\DemoApp"
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\GitHub-Jenkins-Demo>copy index.html "C:\Deployments\DemoApp\index.html"
        1 file(s) copied.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

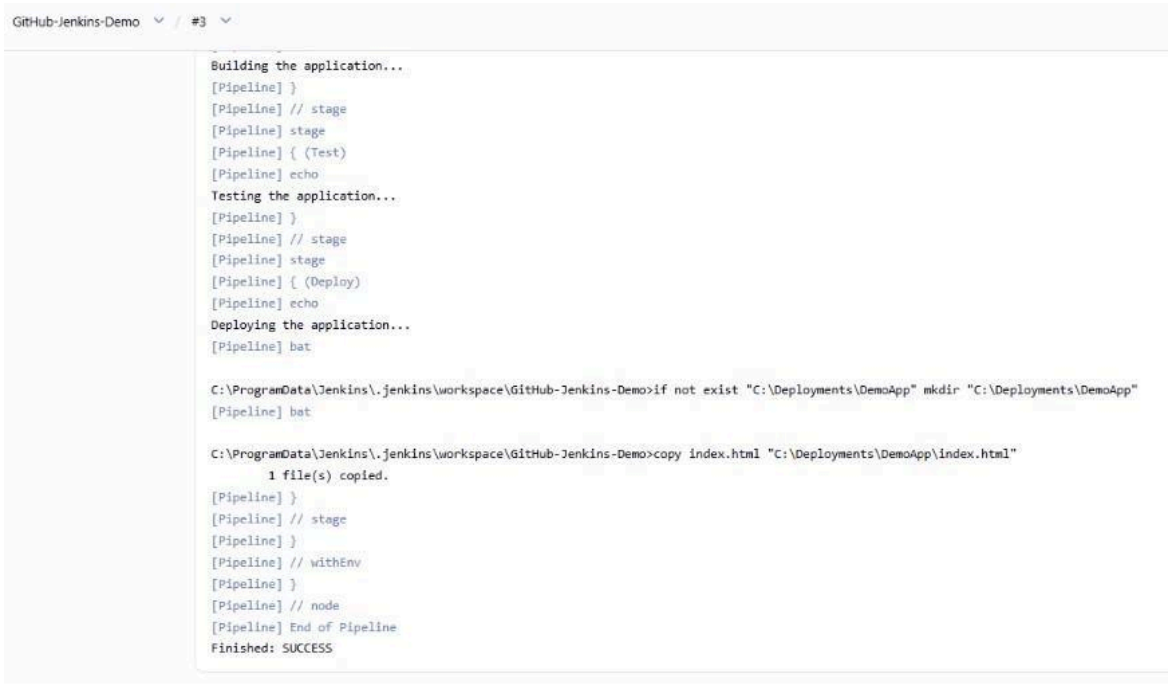


Application Deployed Successfully via Jenkins!




```
<!DOCTYPE html>
<html>
<head>
    <title>CI/CD Demo Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #e3f2fd;
            margin-top: 50px;
        }
        h1 {
            color: #0d47a1;
        }
    </style>
</head>
<body>
    <h1>Application Updated & Deployed via CI/CD Pipeline!</h1>
    <p>Version 2.0 is live.</p>
</body>
</html>
```

Name:Adnan Khan
Roll No:-2401047
AI and DS
Division:T1
Batch:T12
Subject: ASD & DevOps





Conclusion:

The experiment successfully demonstrated an automated CI/CD pipeline using Jenkins and GitHub. Changes made to the application were retrieved from the repository, processed through the defined pipeline stages, and automatically deployed to the target environment. Using a Jenkinsfile made the deployment process version-controlled and reproducible. The successful reflection of updated code in the deployed application showed that automation can reduce manual errors, speed up software releases, and provide consistent application delivery.

LO3 mapped successfully.